DECENTRATHON 6.0 · SOUP KERNEL CHALLENGE · ТРЕК 1

735 МБ вместо 14,3 ГБ

Дообучение Qwen2.5-1.5B через Soup на Kaggle T4 — в 1,5 раза быстрее, в 19,5 раза меньше видеопамяти, то же качество адаптера

Команда **Kuchbrtna** · Nariman Bolatuly · Aidarov Bakhytzhan · Abu Sabytbek · Madiyar Bakhtiyar

ЗАДАЧА

Тот же soup train — быстрее и легче

Что обучаем

Qwen2.5-1.5B в 4 бит (NF4), LoRA r16 на q и v, батч 4, до 512 токенов, 64 шага

Где

Kaggle Tesla T4: 15 ГБ памяти, 65 TFLOPS fp16, PCIe к памяти хоста

Правила

Свои Triton-кернелы через solution.apply(); те же данные, шаги, LoRA и оптимизатор; 10 проверок жюри

База: **77,8 с** и **14 345 MiB** — почти вся память карты.

КУДА УХОДИТ ПАМЯТЬ У БАЗЫ

Модель — меньшая из проблем

Что лежит на GPU	Размер	Наш ответ
Логиты [4 × 512 × 151 936] fp16/fp32	гигабайты	голова без логитов
Активации, сохранённые для backward	до ~2 ГБ	выгрузка на хост
Эмбеддинг и голова в fp32	933 МБ	int8 на хосте
4-битные веса слоёв	~650 МБ	подгрузка по слою
Паддинг: ~43 % позиций батча	лишний счёт	только реальные токены

1 · ГОЛОВА БЕЗ ЛОГИТОВ

Полных логитов не существует

Словарь идёт кусками по 16 384 строки. Один Triton-проход считает loss, энтропию, argmax и градиент (online softmax, как во FlashAttention).

Таблица эмбеддинга и головы хранится в int8 в pinned-памяти хоста и приезжает на GPU двойным буфером, пока считается предыдущий кусок.

head8.py

5,8 → 0,6

ГБ на голову (микробенчмарк), в 1,8 раза быстрее

2 · УДАЛЕНИЕ ПАДДИНГА

Считаем только реальные токены

43 %

позиций батча — паддинг

Все слои работают с упакованными токенами без паддинга. Attention считается отдельно по каждой строке на её реальной длине — батчи не смешиваются.

Attention сохраняет только q и сгруппированные k/v (2 головы вместо 12) и пересчитывается в backward.

3 · НА GPU ТОЛЬКО ТО, ЧТО НУЖНО СЕЙЧАС

Всё остальное ждёт в памяти хоста

Активации · offload.py

Всё, что слой сохраняет для backward, копируется во втором CUDA-потоке в pinned-арену хоста и возвращается на слой раньше в backward. Копии точные и идут параллельно со счётом.

Веса · wstream.py

4-битные веса уходят на хост прямо при загрузке модели. Слой приезжает одной копией на слой вперёд и распаковывается одним запуском кернела.

Плюс загрузка без разогрева аллокатора transformers и без fp32-копии эмбеддинга.

13,6 % времени обучения — наш код

Кернел	Что делает
nf4_unit_dequant_kernel	все 7 весов слоя из NF4 в fp16 за один запуск, бит в бит с bitsandbytes
head_partial_kernel, int8_rows_dequant	голова: loss, энтропия, argmax и градиент по кускам словаря
rms_norm_fwd / bwd	слитый RMSNorm, вход для backward в fp16
add_rmsnorm_fwd / bwd	residual add и RMSNorm одним запуском
rope_rotate_kernel	RoPE для q и k, forward и backward

КАК МЫ ИСКАЛИ

34 версии, проверенные прогонами как у жюри

Версия	Что добавили	Время	Пик памяти
база	Soup как есть	77,8 с	14 345 MiB
v8	удаление паддинга, attention по строкам	55,7 с	4 453 MiB
v13	выгрузка активаций, веса на хосте	58,0 с	1 585 MiB
v22	attention с пересчётом, int8-голова на хосте	56,2 с	825 MiB
v28	арена по размеру, быстрые хуки	53,3 с	629 MiB
v34	слой одним запуском, меньше синхронизаций	50,2 с	735 MiB

Отчёт «где растёт пик» в логе каждого запуска показывал, что закрывать дальше. Между v28 и v34 выбрали скорость: по формуле баллов секунда весит больше 100 MiB.

РЕЗУЛЬТАТ · ПУБЛИЧНЫЙ ПРОГОН KAGGLE T4, --repeats 2

19,5×

меньше памяти
14 345 → 735 MiB

1,52×

быстрее
77,8 → 51,1 с

–0,02 %

eval loss
2,04961 → 2,04927

ALL CHECKS PASSED: все 10 проверок жюри, 57 тестов решения и 79 тестов Soup

ЧЕСТНО О ГРАНИЦАХ

Три приближения, остальное точно

Приближения

- int8-таблица головы и эмбеддинга (loss 1-го шага +0,18 %)
- входы нормализаций для backward в fp16
- маски dropout LoRA на упакованных токенах (то же p = 0,05)

Куда дальше

- матричные умножения пока cuBLAS: ~0,13 с шага
- своя attention с переменной длиной
- int8-умножения и меньше копий через PCIe

github.com/BAITC-Hacks/hack-0e0f2cfa-kuchbrtna · kaggle.com/code/narimanbolatuly/notebook0627412735